

Skill &amp; Product Library / Data Quality

Data Pipeline

Live in production

# Prospect Enrichment Pipeline

A Python enrichment pipeline that guarantees an outsourced SDR agency never calls the same prospect twice.



0% overlap against thousands of already-called contacts; 91% phone coverage

Python

Amplemarket

Clay

data enrichment

SDR

dedup

## Overview

A global HR-tech (EOR) platform pays an outsourced sales agency for SDR calls — so every stale or already-called contact in a send list burns real money and brand equity. This nine-script Python pipeline merges multiple prospecting-tool exports into one master workbook, dedupes against every contact the agency has ever called, enriches missing emails and phones via a data-enrichment API, scrapes public job boards for international-hiring intent, buckets contacts by ICP role, and emits dated weekly send plans — tier-ranked so the best contacts go first. Result: hundreds of unique, confirmed-fresh contacts delivered with zero overlap against thousands of already-called records.

## The Problem

Prospect data lived in five separate enrichment-tool exports with overlapping people, missing emails, and no memory of who the agency had already dialed. Re-feeding called contacts wastes paid SDR time; sending recruiters or benefits admins wastes it twice. The pipeline had to make freshness and ICP fit structural, not a manual QA step.

## How It Works

1. **Build the exclusion memory** — a script parses historical call lists into an exclusion index: thousands of unique keys across three identities (name+company, email, LinkedIn URL). A triple-key dedupe, meaning a contact is caught even if only one identifier matches.
2. **Merge & dedupe** the five source lists (intent-based hot leads, director-level contacts, global-mobility contacts, fallback CEOs, CEO contacts) into one combined workbook, preserving every source label on merge instead of overwriting.
3. **Enrich emails** — a script fills missing work emails via a data-enrichment API's email-finder endpoint (LinkedIn-URL lookup first, name+domain fallback); a follow-up script folds results back in, highlighted for review.
4. **Find net-new contacts** — a people-search script searches the enrichment API per company for target titles (Director+, HR, Ops, CEO, CFO, COO, Global Mobility), with per-role, per-company caps to control credit burn, excluding both the workbook and the already-called list.
5. **Scrape hiring intent** — a script hits public ATS APIs (the job-board platforms companies actually post openings on) and pulls open international roles, a direct buying signal for an Employer-of-Record provider.
6. **Bucket & plan** — contacts are classified into ICP role tabs with a title-based exclusion list (recruiters, benefits admins, EAs never sent), phones deduped by last-10 digits, and rolled into dated weekly send plans: a fixed contact count per week, top tier (email AND phone) first.
7. **Log every send** — send briefs and a send log in the repo track exactly what went to the agency and when.



## Stack & Integrations

| Tool                                                                  | Role                                          | Access needed                         |
|-----------------------------------------------------------------------|-----------------------------------------------|---------------------------------------|
| Python 3 (9 scripts, single orchestrator)                             | Merge, dedupe, classify, batch                | Local machine; standard pip packages  |
| Data-enrichment API (Amplemarket)                                     | Email finder, phone reveal, people search     | API key                               |
| Prospecting-tool exports (Clay)                                       | Source exports (5 lists) + re-enrichment loop | CSV exports; no API needed for intake |
| Public ATS APIs (Greenhouse, Lever, Workable, Ashby, SmartRecruiters) | International job postings as intent signal   | None — public endpoints               |
| Spreadsheet workbooks                                                 | Master workbook + dated weekly send plans     | Local files                           |
|                                                                       |                                               |                                       |

| Tool          | Role                                 | Access needed                          |
|---------------|--------------------------------------|----------------------------------------|
| CRM (HubSpot) | Downstream import of the master list | Import-ready CSVs produced by pipeline |

## Setup Process

1. Clone the project folder; install the two required Python packages.
2. Set your enrichment-API key in the two scripts that call it.
3. Drop the agency's historical called lists in the folder and run the exclusion-builder script.
4. Place your source export CSVs; run the orchestrator (whole pipeline or a single phase).
5. Review the regenerated workbook and dated weekly send plan; tune the role caps and search dictionaries to your ICP.
6. Run locally (the enrichment/ATS endpoints need open egress), or route calls through a workflow-automation webhook.



## What You Need

- [ ] Enrichment-API account + API key (email finder, phone reveal, people search)
- [ ] Prospecting-tool exports (or any contact CSVs) as source lists
- [ ] Historical "already contacted" lists from your agency/sequences
- [ ] Python 3 with two standard packages
- [ ] Time to deploy: ~1 hour; full pipeline run resumes mid-phase if interrupted

## Results

- **Hundreds of unique contacts across hundreds of companies** with **confirmed 0% overlap** against thousands of already-called keys.
- **~91% mobile-phone coverage**, with a meaningful share carrying 2+ phones.
- **~76% work-email coverage**, with automated backfill of the remaining gap.
- Rolling **weekly send plans of a couple hundred contacts**, tier-ranked (top tier = email AND phone goes first); two dozen non-ICP titles auto-excluded.
- Every send logged — send briefs + send log maintained in the repo on an ongoing basis.

## Challenges & Fixes

- **Same person in multiple exports under different identifiers** → single-key dedupe missed them → triple-key dedupe (name+company, email, LinkedIn) plus phone dedupe by last-10 digits.

- **Enrichment credit burn scaling with list size** → per-role, per-company caps and skip-what-exists logic keep spend predictable.
- **Enrichment-API endpoint shapes vary by plan/version** → scripts try multiple endpoint/payload shapes; first match wins, documented fallback to update.
- **Sandboxed environments block vendor egress** → pipeline runs locally by design; a workflow-automation webhook relay is documented as the sandboxed alternative.

## Demonstrates

Data engineering for GTM · API-based enrichment at scale · cost-capped pipeline design · vendor management through automation.



### STACK & INTEGRATIONS

Python

Amplemarket

Clay

HubSpot

Greenhouse

Lever

Workable

Ashby

SmartRecruiters

### WHAT IT NEEDS

- ✓ Data-enrichment API account + key (e.g. Amplemarket)
- ✓ Prospecting-tool exports or contact CSVs as source lists
- ✓ Historical already-contacted lists from your agency/sequences
- ✓ Python 3 with requests + openpyxl

### DISCUSS THIS BUILD

Happy to walk through the architecture, the failures along the way, or how this could translate to your team.

Email Patrick

© 2026 Patrick Diamitani. Case studies anonymized for public portfolio use — no employer-identifying or client-identifying detail included.

[pdiamitani@atlashxm.com](mailto:pdiamitani@atlashxm.com) · [linkedin.com/in/diamitani](https://linkedin.com/in/diamitani)

